JARVIS MASTERPLAN V6 - HERMES AI OS

Stand: 18. Mai 2026

Status: verbindliche Architektur- und Entwicklungsgrundlage fuer die naechste Ausbauphase.

Quelle: Masterplan V5, Future Hermes Todo / Roadmap Intake, Jarvis UI v1 Design Specification und aktuelle read-only Foundation-Module.

1. Executive Summary

Jarvis entwickelt sich vom lokalen Agenten-Framework zum lokalen AI Operating System mit sichtbarer Runtime, Voice, Status, Control Center, Trading-Analyse, Memory, Skills und Tool-Kontrolle.

Die zentrale Rollenverteilung bleibt:

? Jarvis = UI, Runtime, Voice, Status und Control Center.
? Hermes = Brain, Routing, Learning, Agent Orchestration und Entscheidungslogik.
? Ollama / lokale Modelle = lokale Modellschicht fuer einfache, private und kostensensitive Aufgaben.
? Qwen = lokaler oder providerbasierter Kandidat fuer Coding, Reasoning und strukturierte Aufgaben, sofern ueber sichere Provider-Layer angebunden.
? GPT-5.5 / Cloud Codex = primaerer starker Coding- und Architekturarbeiter fuer komplexe Umsetzung, Reviews und groessere Refactors.
? OpenRouter = Cloud-Fallback fuer limitierte Sessions oder spezielle Modelle, nur ueber explizite Provider-Layer und cost-aware Routing.

Masterplan 6 macht die neuen Foundation-Themen verbindlich sichtbar: Runtime Supervisor, Shared Memory / Multi-PC, Skills System, Skill Generator, MCP / Tool Layer, Research Discovery Agent, Cost Optimization, Reflective Learning, Trading Intelligence und eine kuenftige Multi-Agent Workflow Architecture.

Gradio bleibt ausdruecklich nur Entwickler- und Testoberflaeche. Die finale UI wird als futuristisches lokales AI Control Center geplant.

2. Leitentscheidungen

2.1 Systemrollen

? Jarvis ist das Produkt- und Control-Center-Layer.
? Hermes ist das Brain und steuert Routing, Planung, Learning und Agentenlogik.
? Agenten sind spezialisierte Analyse- und Arbeitskontexte.
? Tools sind kontrollierte Faehigkeiten mit Contract, Scope und Safety Flags.
? Statusmodule sind read-only-first und duerfen keine Runtime-Aktionen ausloesen.

2.2 Local-first, Cloud Fallback

? Lokale Modelle und lokale Runtime haben Prioritaet.
? Cloud wird nur bewusst, sichtbar und kostenbewusst genutzt.
? Externe Provider laufen nur ueber explizite Provider-Layer.
? Provider, Modell, Kostenrisiko und Approval-Status muessen sichtbar sein.
? Keine versteckten Cloud-Aufrufe.

2.3 Hybrid-Codex-Workflow

Codex bleibt Coding Worker, nicht System Brain.

Workflow:

1. Hermes/Jarvis definiert Ziel, Safety und Kontext.
2. Codex setzt begrenzte Coding- oder Dokumentationsaufgaben um.
3. Tests und Diffs werden sichtbar gemacht.
4. Commits und Pushes bleiben menschlich kontrolliert.

Modusregeln:

? ChatGPT-Codex / GPT-5.5 primaer fuer komplexe Architektur-, Debugging- und Multi-File-Aufgaben.
? Fast Mode standardmaessig aus.
? Fast Mode nur fuer grosse Refactors oder dringendes komplexes Debugging.
? OpenRouter nur als Fallback bei Limits oder Spezialbedarf.
? Ollama / local fuer kleine Planung, Klassifikation, Zusammenfassung und wiederholbare Low-Risk-Aufgaben.
? OSS / Qwen bleiben Local-Worker-Kandidaten fuer kleine, klar begrenzte Aufgaben.
? Qwen2.5-Coder und `gpt-oss:20b` werden weiter evaluiert, aber nicht als verlaessliche Basis fuer komplexe Agenten-Workflows behandelt.
? Zwei Codex-Fenster duerfen nicht gleichzeitig dieselben Dateien bearbeiten.

2.4 Safety First

? `human_review_required` bleibt Grundregel fuer riskante Aktionen.
? `no_auto_trading` ist fuer Trading dauerhaft sichtbar.
? Keine autonomen Codeaenderungen durch Learning, Research oder Reflection.
? Keine automatischen Installationen.
? Keine Secrets in Dokumentation, Logs, Memory oder Statusausgaben.
? Runtime, Logs, Cache und Secrets bleiben lokal.

3. Zielarchitektur

Zielbild:

text
User
  -> Jarvis UI / Voice / Control Center
  -> Hermes Brain
  -> Routing / Planner / Agent Orchestration
  -> Skills / Tools / MCP Gateway
  -> Runtime Supervisor / Scheduler
  -> Memory / Learning / Shared Approval Layer
  -> Read-only Status / Audit / Approval Queue
code

Verantwortlichkeiten:

? Jarvis UI: Home Dashboard, Chat, Voice, Control Center, Status, Approval UX.
? Jarvis Runtime: kontrollierte Ausfuehrungsumgebung, Health, Logs, Lifecycle.
? Hermes Brain: Intent, Domain, Confidence, Safety, Routing, Agent-Auswahl.
? Hermes Learning: Reflexion, Pattern-Kandidaten, Routing Hints, Skill Vorschlaege.
? Runtime Supervisor: Heartbeat, Jobs, Retry Budget, Resource Limits, Zombie Protection, Cleanup und Health Checks.
? Tool Layer: standardisierte Contracts, Permission Scope, MCP-Faehigkeit, read-only Tools zuerst.
? Memory Layer: lokale Runtime Learnings, strukturierte Memory-Daten, human-readable Obsidian-Wissen und spaeter approved shared memory.

4. Aktueller Stand

Bereits vorhanden oder vorbereitet:

? Hermes Router und Hermes Brain Status.
? Agent Dashboard mit bekannten und geplanten Agenten.
? Runtime Status und System Snapshot.
? Zentrale UI-Statusaggregation.
? Learning / Memory Status fuer lokale `.hermes/` Strukturen.
? Voice Status als read-only Foundation.
? Trading Panel Status und Trading Intelligence Status als read-only Foundation.
? Runtime Supervisor Status Foundation.
? Shared Memory / Multi-PC Status Foundation.
? Skills Status Foundation.

? Skill Generator Status Foundation.
? MCP / Tool Standardization Foundation.
? Research Discovery Status Foundation.
? Cost Optimization Status Foundation.
? Reflective Learning Status Foundation.
? Gradio UI als Entwickler- und Testoberflaeche fuer Statuspanels.
? Jarvis UI v1 Design Specification als Zielbild fuer die spaetere finale UI.

Nicht vorhanden oder noch nicht aktiv:

? Keine echten Background-Loops.
? Kein produktiver Scheduler.
? Keine autonome Skill-Ausfuehrung.
? Keine echte Multi-PC-Synchronisation.
? Keine echten Reddit-, GitHub-, arXiv- oder Web-Research-Jobs.
? Keine MCP-Server oder MCP-Clients in produktiver Nutzung.
? Keine Brokerverbindung.
? Keine Orders.
? Kein Auto-Trading.
? Keine finale React/Tauri/FastAPI-Oberflaeche.

5. UI-Zielbild

Gradio ist nur Dev/Test UI.

Die aktuelle Gradio UI bleibt sinnvoll fuer:

? manuelle Statuschecks
? JSON-Inspektion
? Validierung der Foundation-Module
? sichere Entwickler-Tests

Sie ist nicht:

? finale Jarvis UI
? Produktions-UX
? langfristiges Control Center
? visuelle Designbasis

Finales Ziel:

Jarvis UI v1 wird ein futuristisches lokales AI Control Center:

? dunkel
? modern
? animiert
? modular
? hochwertig
? AI-first
? Jarvis/Iron-Man-inspiriert
? Control-Center-Feeling

Hauptbereiche:

? Home Dashboard
? Chat / Gespraech
? Hermes Brain Panel
? Agent Dashboard
? Runtime Control
? Voice Interface
? Trading Panel
? Taskline / Activity Feed
? Learning & Memory
? Developer Debug
? Skills / Tools
? Research Discovery
? Cost Optimization

Permanent sichtbar oder schnell erreichbar:

? XAUUSD Livekurs
? EURUSD Livekurs
? GER40 spaeter
? Wetter
? aktive Agenten
? laufende Tasks
? Hermes Status
? Ollama Status
? Runtime Warnings
? Trading Signals
? Trading Setup Watch / Trigger-Status
? Provider / Model Status
? `no_auto_trading`
? Approval Queue

Layout-Ziel:

? links: Agent Activity / Taskline
? mitte: Chat, Voice und Hauptinteraktion
? rechts: Hermes Brain, Trading und Model Routing
? unten: Runtime, Logs, Memory und Systemstatus

Technikoptionen fuer spaeter:

? React / Vite
? Tauri
? FastAPI
? WebSocket / Event Stream

Diese Optionen sind noch keine Implementierungsentscheidung. Masterplan 6 legt nur Zielbild und Sicherheitsanforderungen fest.

6. Runtime-Zielbild

Der Runtime Supervisor wird die Kontrollinstanz fuer geplante Hintergrund- und Agentenaufgaben.

Geplante Funktionen:

? Heartbeat
? Scheduler / Cron-Struktur
? Cron als Agent-Jobs statt reine Shell-Crons
? Background Jobs
? Agent Lifecycle
? Zombie Protection
? Context Lifecycle
? Context Compression
? Resource Limits
? Runtime Cleanup
? Health Checks
? Hallucination Gate
? Retry Budget / `max_retries` pro Task
? 5-Minuten-cTrader-QUOTE-Checks spaeter

Grundregeln:

? Keine heimlichen Services.
? Keine unkontrollierten Loops.
? Jeder Job hat Zweck, Owner, Limits, Retry Budget und Safety Flags.
? Schreibende Jobs brauchen Review.
? Research Jobs bleiben read-only.
? Runtime-Aktionen werden in der finalen UI sichtbar und bestaetigungspflichtig.

Geplante Jobs:

? Reddit Research Scan
? GitHub Trend Scan
? Weather Refresh
? cTrader Quote Check

? Prediction Feedback Check
? Memory Cleanup Review

7. Memory / Learning

Memory wird in drei Schichten getrennt:

? `.hermes/` = Runtime Learning Layer, lokal und laufzeitnah.
? `memory/` = Structured Memory Layer fuer strukturierte, freigegebene Inhalte.
? `obsidian/` = Human Knowledge Layer fuer menschlich lesbare Notizen,
Architektur, Entscheidungen und Wissensgraph.

Zusaetzlich geplant:

? `memory_shared/` fuer genehmigte, synchronisierbare Learnings.
? `approved_learnings/` fuer explizit freigegebene dauerhafte Learnings.
? `shared_skills/` fuer freigegebene Skills.
? `routing_hints_approved/` fuer genehmigte Routing-Hints.
? `trading_patterns_approved/` fuer freigegebene Trading-Pattern.

Regeln:

? Keine Orphan- oder Shadow-Stores.
? Single-Store-Design pruefen.
? Runtime-Daten werden nicht unkontrolliert synchronisiert.
? Persistence ist approval-based.
? Lokale Learnings entstehen zuerst lokal.
? Hermes schlaegt dauerhafte Learnings vor.
? Frank bestaetigt.
? Erst danach wird shared memory aktualisiert.
? Zweiter PC synchronisiert nur approved memory.

Reflective Learning:

? post_task_review
? pattern_extraction
? failure_analysis
? success_pattern_detection
? skill_candidate_generation
? routing_hint_candidate_generation
? confidence_adjustment_candidate

Reflection erzeugt Vorschlaege, keine direkten Code- oder Runtime-Aenderungen.

8. Trading Intelligence

Trading Intelligence ist Analyse-, Setup-Watch-, Backtesting- und Lernschicht,
keine Order-Schicht.

Unterstuetzte Ziel-Symbole:

? XAUUSD
? EURUSD
? GER40
? US500
? Forex-Majors

Geplante Pipeline:

? cTrader QUOTE Feed geplant
? read-only quotes only
? no trade execution
? Composite Trading Indicator als Feature- / Signal-Engine geplant
? Trading Feature Store geplant
? future feature extraction
? session tagging
? market regime tagging
? continuous backtesting
? strategy evaluation

? signal scoring
? setup watch
? signal alerts only after trigger conditions are met
? entry / exit warnings
? no-trade zones

Rollenmodell:

? Composite Trading Indicator liefert Features, Marktstruktur, Scores und nicht-repaintende Signal-Kandidaten.
? Hermes-Agent bewertet, lernt, vergleicht und entscheidet als lernende Bewertungs- und Entscheidungsinstanz.
? Indicator-Regeln bleiben deterministische Feature-Erzeugung.
? Hermes-Learning bleibt reviewbar und approval-gesteuert.
? Trennung bleibt verbindlich: Indicator liefert Features, Hermes lernt und entscheidet.

Prediction Learning:

? Prediction Feedback Loop geplant
? Prediction Scoring geplant
? Confidence Tracking geplant
? Confidence Calibration geplant
? Outcome Review geplant
? Feature Importance Tracking geplant
? keine autonome Ausfuehrung

Continuous Backtesting:

? historische cTrader-Daten kontinuierlich auswerten.
? XAUUSD, EURUSD, GER40, US500 und Forex-Majors getrennt backtesten.
? automatische Hypothesentests vorbereiten.
? Feature-, Session- und Timeframe-Vergleiche durchfuehren.
? Walk-Forward- und Out-of-Sample-Tests planen.
? Paper-Trading-Phase vor Live-Phasen.
? Live-vs-Backtest-Vergleich fuer spaetere Forward-Auswertung.
? Demo- / Forward-Tests spaeter.
? Modellvergleich XGBoost / LightGBM.
? schlechte Strategien deaktivieren oder zurueckstufen.
? Risiko reduzieren, wenn Performance schwach ist.

Setup Watch / Signal Alerts:

? moegliche Long- und Short-Szenarien frueh erkennen.
? Vorwarnung: "Setup koennte entstehen."
? Zeitfenster typischerweise 10 bis 30 Minuten.
? Trigger-Bedingungen, Entry-Zone, Confidence, Stop-Loss-Vorschlag, Take-Profit- / Zielzonen und Invalidation-Level sichtbar machen.
? Entry- und Exit-Warnungen sichtbar machen.
? No-Trade-Zonen sichtbar machen.
? Statusmodell: `watching`, `armed`, `triggered`, `expired`.
? Signal erst ausloesen, wenn Bedingungen eintreten.
? Signal-Kandidaten duerfen nicht repainten und gelten nur nach Kerzenschluss.
? Entscheidung bleibt bei Frank.

Geplante Modelle:

? XGBoost
? LightGBM
? future transformer experiments
? ensemble later optional

Feature Engine:

? Trading Feature Store
? Composite Indicator Features
? Session Features London / New York
? Volatility Features
? Momentum Features
? Spread Tracking

? Time Features
? News spaeter optional
? Feature Importance
? Higher Timeframe Alignment
? Market Structure
? Rejection Quality
? Pattern-Cluster statt starre Regeln

Market Regime / Strategy Library:

? Trendmarkt, Seitwaertsmarkt, hohe Volatilitaet, News-Markt und illiquide Marktphasen erkennen.
? Spread- / Liquiditaetsfilter und Session-Fokus London / New York.
? Strategie-Kandidaten: Trend Pullback, Breakout, Mean Reversion und No-Trade-Filter.
? Strategie-Bewertung pro Marktregime.
? Gold, Indizes und Forex getrennt behandeln.
? Pattern-Cluster koennen mehrere Setups zusammenfassen, ersetzen aber keine Review- und Safety-Gates.

cTrader / C# Zielarchitektur:

? Core: gemeinsame Berechnungen, Feature-Definitionen und Signal Contracts.
? Indicator: Composite Trading Indicator fuer Visualisierung und Feature-Ausgabe.
? Bot: spaetere kontrollierte Ausfuehrungsschicht, aktuell deaktiviert.
? Analyzer: Backtest-, Feature- und Ergebnisanalyse.
? Research: Overnight Research, Hypothesentests und Report-Erzeugung.

Overnight Research Mode:

? historische oder gespeicherte Daten testen.
? keine Orders.
? keine Broker-Trade-Verbindung.
? morgens Report erzeugen.
? Lernkandidaten zur Freigabe vorbereiten.
? Frank entscheidet, was dauerhaft gelernt wird.

Trading Agent Modularisierung:

? Market Data Agent
? Market Regime Agent
? Setup Watch Agent
? Signal Scoring Agent
? Risk Agent
? Backtesting Agent
? Prediction Review Agent
? News Context Agent
? Research Agent
? Hermes bleibt Orchestrator.

Safety:

? `no_auto_trading: true`
? `no_trade_execution: true`
? `human_review_required: true`
? Broker-Verbindung bleibt deaktiviert bis explizite Freigabe.
? TRADE-Verbindung bleibt deaktiviert bis explizite Freigabe.
? QUOTE ist read-only Marktdatenquelle.
? Modelle duerfen Prognosen bewerten, aber keine Orders ausloesen.
? Human Approval bleibt Pflicht.
? Kein Martingale, kein Grid, keine Risikoerhoehung nach Verlust.
? Keine ungetesteten Regeln live einsetzen.
? Keine automatische Risikoerhoehung aus Backtest- oder Learning-Ergebnissen.
? Kill Switch, Drawdown Limits, Tagesverlustlimit, Wochenverlustlimit und Audit Log sind spaetere Pflicht-Gates vor jeder Live-Phase.
? Risiko pro Trade 0,25-1 % nur als spaetere Richtlinie.

UI-Bezug:

? Trading Learning / Backtest Center fuer Backtest Runs, Strategy Comparison,
  Prediction Feedback, Setup Watch Results und Confidence Calibration.
? Jarvis Learning UI fuer Prediction -> Outcome -> Bewertung -> Learning.
? Setup Watch Panel fuer Entry, Exit, Confidence, SL/TP, Invalidation und
  No-Trade-Zonen.
? Approval Queue fuer dauerhafte Trading-Learnings.

Autonomy Roadmap:

1. `analysis_only`
2. `setup_watch`
3. `signal_alerts`
4. `prediction_feedback`
5. `continuous_backtesting`
6. `paper_trading`
7. `demo_trading`
8. `micro_live_with_approval`
9. `approval_required_live_trading`
10. `later_optional_autotrading`

Vollautomatik bleibt nur eine spaetere Option.

9. Tool / Skill / MCP

9.1 Skills System

Skills sind wiederverwendbare Markdown-Playbooks und prozedurales
Gedaechtnis.

Geplante Kategorien:

? architecture
? debugging
? trading
? runtime
? ui
? codex_workflows
? deployment
? research

Skill Registry:

? versioning_required
? metadata_required
? owner_required
? safety_flags_required
? review_required
? connector_scope_required_later

Workflow:

1. proposed
2. reviewed_by_frank
3. approved
4. active
5. deprecated

Generated Skills werden niemals automatisch aktiv.

9.2 Skill Generator

Der Skill Generator bereitet aus Tool- und API-Spezifikationen Skill-Entwuerfe
vor, fuehrt sie aber nicht aus.

Moegliche Quellen:

? Apify Actors
? MCP Tools
? lokale CLI-Tools
? cTrader QUOTE Bridge

? Weather Provider
? Reddit / GitHub Research Tools
? OpenAPI Specs

Ausgaben:

? Skill-Dokumentation
? Input Schema
? Execution Contract
? Test Prompts
? Usage Notes
? Safety Flags
? Rate-Limit-Metadaten
? Cost-Metadaten

Regeln:

? keine Auto-Aktivierung
? `human_review_required`
? secrets_never_embedded
? pagination_required
? truncation_handling_required
? structured_output_required

9.3 MCP / Tool Layer

MCP ist als Standardisierungs- und Gateway-Idee gesetzt.

Strategie:

? Hermes spaeter als MCP Client denkbar.
? Hermes spaeter als MCP Server denkbar.
? MCP Gateway geplant.
? Read-only Tools zuerst.
? Keine Toolausfuehrung ohne Review.
? Skills + Connectors + Sub-Agents + Workflows als zukuenftiges Hermes-Pattern.
? Agent Chains und Tool-/Skill-Flows muessen im Jarvis Control Center sichtbar werden.

Tool Registry:

? metadata_required
? owner_required
? versioning_required
? safety_flags_required
? permission_scope_required

Connector / Sub-Agent Registry spaeter:

? Skill Registry
? Tool Registry
? Connector Registry
? Approval Gates
? Audit Logs
? Safety Flags
? Versionierung
? Multi-PC-Synchronisation nur fuer gepruefte Skills

Geplante Tool-Kategorien:

? filesystem_readonly
? browser_assist
? voice_runtime
? weather_provider
? ctrader_quote
? reddit_research
? github_research
? obsidian_knowledge
? memory_retrieval
? runtime_status

Safety:

? read_only_default
? write_requires_approval
? external_api_requires_review
? secrets_never_exposed
? trade_execution_disabled
? audit_log_required_later

10. Research Discovery Agent

Hermes soll externe Ideen als kuratierte Vorschlaege sammeln. Research bleibt read-only.

Quellen:

? Reddit
? GitHub
? arXiv
? MCP ecosystem
? Hermes-agent ecosystem
? Ollama / OpenRouter news
? Trading AI / cTrader topics

Monitored Topics:

? LangGraph
? CrewAI
? AutoGen
? OpenClaw
? SWE-Agent / OpenDevin
? MCP
? local models
? agent memory
? scheduler / runtime supervisor
? skill systems
? trading ML

Pipeline:

1. scan
2. deduplicate
3. summarize
4. extract_ideas
5. score_relevance
6. propose_for_review
7. archive_or_promote

Safety:

? read_only_research
? no_auto_code_changes
? no_auto_installations
? human_review_required
? cite_sources_required
? Quelle und Datum dokumentieren
? Lizenzlage markieren

External Pattern Review:

? `https://github.com/wondelai/skills` als Inspiration fuer Markdown-Playbooks.
? `https://github.com/builderz-labs/mission-control` als Inspiration fuer Agent Fleet View, Task Dispatching, Session Monitoring, Cost Tracking, Logs / Audit Trail, Event Store und Live Telemetry.
? Anthropic Financial Services, Legal und Life Sciences Pattern als spaetere Referenz fuer spezialisierte Agenten, Skills, Data Connectors und MCP-nahe Forschungs- oder Analysewerkzeuge pruefen.
? Nichts direkt kopieren.
? Lizenz pruefen.

? Nur Patterns extrahieren.
? Keine fremden Skills automatisch aktivieren.
? Keine Repositories klonen ohne explizite Freigabe.

11. Multi-PC Sync

Ziel: Zwei oder mehr PCs koennen dieselben freigegebenen Erfahrungen und Dokumente nutzen, ohne lokale Runtime, Logs, Cache oder Secrets zu mischen.

Synchronisierbar:

? Code und Dokumentation ueber GitHub.
? Obsidian- und Wissensnotizen, sofern bewusst freigegeben.
? Approved Learnings.
? Approved Skills.
? Approved Routing Hints.
? Approved Trading Patterns.

Lokal-only:

? `.hermes/runtime`
? `runtime/`
? `logs/`
? `.env.local`
? cache
? local model files
? Secrets

Rollen:

? primary_pc
? secondary_pc
? offline_mode
? conflict_handling: manual_review_required

Regeln:

? Keine unkontrollierte Runtime-Synchronisation.
? Kein Sync von `.env.local`.
? Kein Sync von Secrets.
? Kein Sync roher Logs.
? Konflikte werden manuell reviewt.
? Cross-device Memory Sync Engine ist spaeter, nicht jetzt.

12. Cost Optimization

Ziel: Provider- und Modellnutzung wird transparent, aufgabenabhaengig und kostenbewusst gesteuert.

Codex-Strategie:

? ChatGPT-Codex primaer.
? Fast Mode standardmaessig aus.
? Fast Mode nur fuer grosse Refactors, Multi-File-Architektur oder dringendes komplexes Debugging.
? OpenRouter als Fallback bei Limits.
? Ollama / local fuer kleine Aufgaben.
? Kleine Doku-, Planungs- und Status-Foundation-Aufgaben ohne Fast Mode.

Provider-Prioritaet:

? local_ollama_first
? chatgpt_codex_for_complex_code
? openrouter_fallback_for_limited_sessions
? manual_review_for_costly_tasks

Cost Controls:

? credit_monitoring_required
? no_hidden_cloud_calls

? provider_logged
? model_logged
? estimated_cost_later
? human_review_for_expensive_tasks

Future Dashboards:

? Codex usage panel
? OpenRouter credit panel
? model routing history
? provider cost summary
? local / cloud ratio

13. Safety

Globale Sicherheitsregeln:

? Human review bleibt Pflicht fuer riskante Aenderungen.
? Keine automatischen Commits.
? Keine automatischen Pushes.
? Keine unreviewed production config changes.
? Keine Secrets lesen, speichern oder anzeigen.
? Keine Runtime-Schreibzugriffe durch Statusmodule.
? Keine autonome Code-Selbstmodifikation.
? Keine automatische Dependency-Installation.
? Keine Tool- oder Skill-Ausfuehrung ohne Review.
? Keine automatischen Trades.
? Keine Brokerverbindung ohne explizite Freigabe.

UI-Safety:

? Approval Requests sichtbar.
? Keine versteckten Actions.
? `no_auto_trading` sichtbar.
? Cloud-Kosten sichtbar.
? Aktive Provider / Modelle sichtbar.
? Runtime-Aktionen confirmieren.
? Tool- und Skill-Ausfuehrung nur mit Review.

Trading-Safety:

? `no_auto_trading`
? `human_review_required`
? `no_trade_execution`
? TRADE deaktiviert
? QUOTE read-only
? keine ungetesteten Regeln live
? kein Martingale
? kein Grid
? keine Risikoerhoehung nach Verlust
? Kill Switch, Drawdown Limits, Tagesverlustlimit, Wochenverlustlimit und Audit Log vor Live-Phasen verpflichtend planen

Research-Safety:

? read-only Recherche.
? Keine autonomen Codeaenderungen.
? Keine autonomen Installationen.
? Keine ungeprueften Empfehlungen direkt uebernehmen.
? Quellen und Datum dokumentieren.

14. Masterplan-6-Roadmap

MUST

? Trading Setup Watch verbindlich fuer die naechste Roadmap ausarbeiten.
? Trading Prediction Feedback Loop spezifizieren.
? Trading Continuous Backtesting fuer XAUUSD, EURUSD und GER40 planen.
? Trading Safety Gates konkretisieren.

? Jarvis Learning UI mit Trading Feedback, Approval Center und sichtbaren Learnings verbinden.
? Runtime Event Standardisierung fuer Agent-Flows, Trading-Status und Approval Events fortfuehren.
? `no_auto_trading` / `human_review_required` dauerhaft sichtbar halten.
? Multi-Agent Workflow Architecture entwerfen.
? Multi-PC Shared Learning verbindlich entwerfen.
? Runtime Supervisor Foundation zu einer kontrollierten Supervisor-Struktur weiterentwickeln.
? Memory Architecture mit klarer Store-Trennung finalisieren.
? Research / Discovery Agent als read-only Pipeline planen.
? Cost-aware Codex / OpenRouter / Ollama Strategy im UI sichtbar machen.
? Trading `no_auto_trading` Safety dauerhaft im UI und Status fuehren.
? External Pattern Review fuer Wondel.ai Skills und Mission Control dokumentiert durchfuehren.

SHOULD

? Market Regime Detection planen.
? Strategy Library entwerfen.
? Signal Score System definieren.
? Risk Agent spezifizieren.
? Fish Audio / Fish Speech Evaluation fuer spaetere Voice Runtime pruefen.
? Unsloth / Local Fine-Tuning Evaluation vorbereiten.
? Anthropic Plugin Pattern Review dokumentieren.
? Skill / Connector / Sub-Agent Pattern konkretisieren.
? Hermes Skills System ausbauen.
? Skill Registry definieren.
? Skill Review Workflow konkretisieren.
? Skill Generator als Draft-Generator spezifizieren.
? MCP Gateway planen.
? Agent Dashboard erweitern.
? Reflective Learning Phase als Approval-Queue-Kandidat ausbauen.
? Obsidian Knowledge Integration pruefen.

LATER

? Paper Trading.
? Demo Trading.
? Micro Live Trading mit Approval.
? Optional Autotrading.
? Local Fine-Tuning.
? Voice Personality Layer.
? Full MCP Connector Marketplace.
? Automatische Skill-Generierung aus Apify / MCP.
? WebSocket Live Runtime.
? Cross-device Memory Sync Engine.
? Agent-to-Agent Consensus.
? Advanced Token Dashboard.
? Trading ML Training Pipeline.
? Event Store / SQLite fuer Audit und Live-Telemetry pruefen.
? Messaging Gateway optional: Telegram, Discord, Slack, Email.

15. Abnahmeliste

Masterplan 6 gilt als akzeptiert, wenn:

? Jarvis und Hermes Rollen eindeutig getrennt sind.
? Local-first und Cloud fallback beschrieben sind.
? Ollama, Qwen, GPT-5.5 / Cloud Codex und OpenRouter Rollen beschrieben sind.
? Hybrid-Codex-Workflow dokumentiert ist.
? Gradio klar als Dev/Test UI markiert ist.
? Finale UI als futuristisches AI Control Center beschrieben ist.
? Runtime Supervisor, Shared Memory, Skills, Skill Generator, MCP / Tool Layer, Research Discovery, Cost Optimization, Reflective Learning und Trading Intelligence enthalten sind.
? cTrader QUOTE Feed als geplant und read-only beschrieben ist.
? `no_auto_trading` und `human_review_required` enthalten sind.

? Obsidian / `memory/` / `.hermes/` Wissensarchitektur beschrieben ist.
? Roadmap mit MUST, SHOULD und LATER enthalten ist.
? Keine Implementierung, Runtime-Aenderung, Services, Secrets, Commits oder Pushes Teil dieses Masterplans sind.

16. Uebergabe-Prompt fuer neuen Chat

text
Arbeite im Projekt ~/jarvis.

Nutze docs/Masterplan/Jarvis_Masterplan_V6_Hermes_AI_OS.md als aktuelle verbindliche Architektur- und Entwicklungsgrundlage.

Wichtige Rollen:
? Jarvis = UI / Runtime / Voice / Control Center.
? Hermes = Brain / Routing / Learning / Agent Orchestration.
? Codex = Coding Worker, nicht System Brain.

Grundregeln:
? Local-first, Cloud nur als bewusster Fallback.
? Gradio bleibt Dev/Test UI.
? Finale UI ist ein futuristisches lokales AI Control Center.
? Statusmodule sind read-only-first.
? Keine Runtime-Dateien ohne expliziten Auftrag aendern.
? Keine Secrets lesen oder speichern.
? Keine Services starten, ausser explizit verlangt.
? Keine Commits oder Pushes.
? no_auto_trading bleibt Pflicht.
? human_review_required fuer riskante Aktionen.

Aktuelle V6-Schwerpunkte:
? Runtime Supervisor
? Shared Memory / Multi-PC
? Memory Architecture
? Skills System
? Skill Generator
? MCP / Tool Layer
? Research Discovery Agent
? Cost Optimization
? Reflective Learning
? Trading Intelligence mit cTrader QUOTE Feed geplant

Wenn du Code aenderst:
? klein und reviewbar arbeiten
? bestehende Architektur respektieren
? Tests ausfuehren, wenn relevant
? geaenderte Dateien und Diff zusammenfassen
code

17. Nicht-Ziele

Dieser Masterplan erzeugt nicht:

? PDF
? React / Tauri / FastAPI-Code
? neue Services
? Scheduler oder Background-Loops
? Brokerverbindungen
? Orders
? echte Web-, Reddit-, GitHub- oder arXiv-Abfragen
? API-Keys oder Secrets
? Commits oder Pushes

Masterplan 6 ist Dokumentation und Architekturgrundlage.